

0.26. En diagonale : deplace

`deplace(x, y, tuile, pasX, pasY)` : les deux axes en une ligne. Avec 5 et 5, la lettre descend en biais.

1. Ce qu'il faut comprendre

Une seule ligne pour les deux axes : `deplace`.

Ce qui est nouveau ici : `deplace(colonne, ligne, tuile, pasX, pasY)`. Elle a **deux** nombres de pas : `pasX` pour l'horizontale (+ droite, - gauche) et `pasY` pour la verticale (+ bas, - haut).

La diagonale : avec 5 et 5, la lettre fait à chaque quart de seconde un pas sur X **et** un pas sur Y **en même temps** : elle descend en biais, (1, 1), (2, 2)... jusqu'à (5, 5). C'est impossible avec `deplace_x` puis `deplace_y`, qui font un axe après l'autre.

Deux positions à ranger : une fonction ne rend qu'une valeur. `deplace` rend la colonne, et dépose la ligne dans une variable de la console. **Seule sur sa ligne**, la console range les deux : `x` **et** `y`. Écris-la toujours ainsi : `x = deplace(...)` ne rangerait que `x`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace>` → [0.25.3. La fonction deplace\(\) — avancer sur les deux axes](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1209 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	62 o	← la console, à l'allumage — la mise en route de la console
deplace()	236 o	← main() — une fonction fournie par le langage
main()	56 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles — la police entière

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```

poser()   × 2    26 o sur place, dans deplace() + AdresseCase (10 o)
nombre()  × 2    20 o sur place, dans main()   + EcrireNombre (33 o)
effacer() × 1    15 o sur place, dans deplace() + AdresseCase (10 o), EffacerCases (12 o)
image()   × 2     6 o sur place, dans deplace(), main() + AttendreImage (28 o)
Tes fonctions appelées :
deplace() × 1    23 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

#include <deplace>    // fait avancer une tuile sur les deux axes
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <nombre>     // écrit un nombre en chiffres

```

2. UNE VARIABLE — X

```

uint8_t x = 0;      // la COLONNE de la lettre

```

3. UNE VARIABLE — Y

```

uint8_t y = 0;      // la LIGNE de la lettre

```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {
    // La ligne, morceau par morceau :
    //
    //   deplace(x, y, ALPHABET[0], 5, 5);
    //           | | |           | |
    //           | | |           | +-- pasY : +5, vers le bas
    //           | | |           +----- pasX : +5, vers la droite
    //           | | +----- tuile : la lettre A
    //           +---+----- x, y : d'où elle part ; la console y
    //                           range l'arrivée : x = 5, y = 5
    //
    //
    //   A           (0, 0)
    //   ↘
    //   ↘           à chaque pas : +1 sur X ET +1 sur Y
    //   ↘
    //   ↘
    //   A           (5, 5)
    deplace(x, y, ALPHABET[0], 5, 5);
    nombre(0, 17, x);    // 005
    nombre(4, 17, y);    // 005

    while (true) {
        image();
    }
}

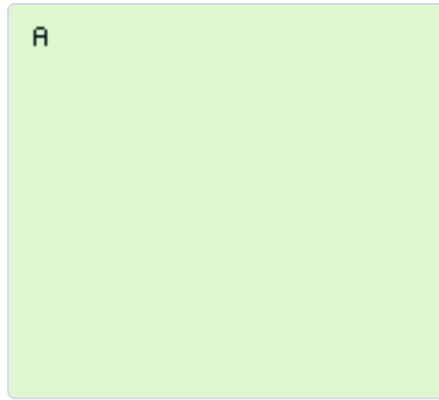
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



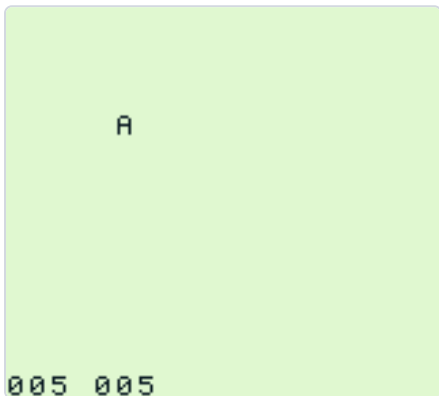
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui descend en diagonale, du coin en haut à gauche jusqu'en (5, 5). En bas, 005 et 005.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ en biais : (1, 1), (2, 2), (3, 3), (4, 4)

✓ jamais tout droit : ni (3, 0), ni (0, 3)

✓ x et y rangés tout seuls : 5 et 5

✓ le A est en (5, 5) (5,5)

Ensuite : **0.26.1. En diagonale : deplace — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++